

SnapT — Notification Trigger Mapping

Drafted: July 26, 2026 **Status:** Working reference document. Not yet implemented — this is the source of truth for building Firebase Cloud Messaging (push) and the in-app Notifications inbox logic.

How to read this

- **Push** = device-level notification via Firebase Cloud Messaging, delivered even when the app is closed.
 - **In-app** = appears in the Notifications inbox inside the app, no device push.
 - Default rule applied throughout: push is used when the event is time-sensitive, requires action, or involves money/safety. In-app only is used when the event is informational and can wait until the user next opens the app. Internal events never push to end users at all — they route to the Admin Portal instead (see Section 7).
 - Rows marked [**JUDGMENT CALL**] are places I made a reasonable default without an explicit decision from you — check these specifically.
-

1. Booking lifecycle

Trigger	Recipient	Push	In-app	Notes
Booking confirmed	Client + Creator	✓	✓	Immediate confirmation both sides need
Creator assigned/matched	Client	✓	✓	If matching is instant at booking time, this may merge with "Booking confirmed" — confirm once matching latency is known
Session reminder — 24 hours out	Client + Creator	✓	✓	
Session reminder — 2 hours out [JUDGMENT CALL]	Client + Creator	✓	✗	Not in your original list — added as best practice; a same-day reminder meaningfully reduces no-shows and is worth the extra push
Creator cancelled booking	Client	✓	✓	Time-sensitive, ties directly into refund/rematch flow
Client cancelled booking	Creator	✗	✓	[JUDGMENT CALL] No action required from creator, just frees their slot — in-app is sufficient
Booking rescheduled (new time confirmed)	Client + Creator	✓	✓	

2. Session day

Trigger	Recipient	Push	In-app	Notes
Creator "on the way"	Client	✓	✗	[JUDGMENT CALL] Purely live/time-sensitive, no lasting value in the inbox afterward
Creator checked in / session started	Client	✓	✓	
Session ended	Client + Creator	✗	✓	[JUDGMENT CALL] Informational, sets next-step state (editing/processing) but not urgent

3. Delivery & revisions

Trigger	Recipient	Push	In-app	Notes
Footage submitted by creator	Internal only	—	—	Routes to Editing Workspace queue, not client-facing at all
Edit ready / delivered	Client	✓	✓	This is the payoff moment — always push
Revision requested	Creator	✓	✓	Actionable, creator needs to respond
Revision delivered	Client	✓	✓	

4. Money

Trigger	Recipient	Push	In-app	Notes
Payment charged (booking confirmation receipt)	Client	✗	✓	[JUDGMENT CALL] Routine, expected, receipt-style — no need to interrupt
Cancellation/reschedule fee charged	Client	✓	✓	[JUDGMENT CALL] Unlike a routine charge, this is often unexpected to the user in the moment — push for transparency, avoid "surprise charge" support tickets
Refund issued	Client	✓	✓	People actively wait on refunds — push reduces anxiety and support volume
Earnings available for payout	Creator	✗	✓	[JUDGMENT CALL] Not time-sensitive, they'll see it next time they check Earnings
Payout sent	Creator	✓	✓	Confirms money actually moved — creators want this pushed

5. Creator application & status

Trigger	Recipient	Push	In-app	Notes
Application submitted	Creator	✗	✓	Confirmation only
Application approved	Creator	✓	✓	Meaningful milestone, they can now receive bookings
Application declined	Creator	✓	✓	Needs to reach them reliably even if they've stopped checking the app
Specialty/profile updated successfully	Creator	✗	✓	Low-stakes confirmation

6. Trust & safety / disputes

Trigger	Recipient	Push	In-app	Notes
Report received (acknowledgment)	Reporting user	✗	✓	[JUDGMENT CALL] Confirms receipt without implying urgency has passed — avoid a push that might feel dismissive right after a safety event
Dispute opened	Both parties	✓	✓	Actionable — evidence window is only 72 hours per the Dispute Resolution Policy, push is necessary
Dispute resolved	Both parties	✓	✓	
Strike issued	Creator	✓	✓	May carry appeal window, needs to reach them reliably
Suspension applied	Creator	✓	✓	Account-impacting, always push, always with reason stated per policy

7. Internal / Admin-only alerts (not user-facing)

These never generate a push or in-app notification to the client or creator involved. They route to the **Admin Portal** instead, and — given the severity spread here — probably need their own escalation tiering, not just a dashboard list.

Trigger	Routed to	Suggested urgency
SOS / safety report filed during active session	Admin Portal, high-priority queue	Immediate — recommend this also triggers an email alert (via Resend) to on-call staff, not just a dashboard entry, given the real-time safety stakes
General "Report a Problem" submission	Admin Portal, standard queue	Standard
Content moderation flag	Admin Portal, moderation queue	Standard, escalates if repeated
Creator reaching 3rd/4th cancellation strike	Admin Portal, creator review queue	Standard
Background check flagged/failed	Admin Portal, vetting queue	Standard

This is worth flagging on its own: the SOS feature we just designed is only as good as what happens after it's filed. A safety report sitting unopened in an admin queue overnight defeats the purpose. Once the Admin Portal is scoped, on-call alerting for this specific category should be treated as a requirement, not a nice-to-have.

8. Not yet decided

- **New booking request to creator** — only needed if creator matching isn't instant/automatic (e.g., if creators can accept/decline rather than being auto-assigned). Current build implies auto-matching, so this may not apply — confirm.
- **Emergency contact share** — this goes to a non-Snapt-user email address, sent via Resend, not a push/in-app notification at all. Snapt does not use SMS anywhere in the product — worth noting so this doesn't get lost in FCM planning specifically.

9. Build implications

- The existing **Notifications inbox** screen (already built) will need to support categorization/filtering once this many trigger types exist — worth a quick design pass so it doesn't become an undifferentiated wall of items.
- Add to backend requirements doc: **notification trigger service** — a central dispatcher that listens for these state-change events and fires the correct push/in-app/email (Resend) combination, rather than notification logic being scattered across individual features as they're built. No SMS channel exists in this product.